

Unit – 1 : Session – 4 : SLO – 2 | Multi-Paradigm Use**Activity (Solutions)**

- 1) OOP + functional:

```
class Greeter { String prefix; Greeter(String p){ prefix=p; } }  
java.util.function.Function<String,String> f = name -> "Hello " + name;  
System.out.println(f.apply("Sam"));
```

 — lambda used alongside a class.
- 2) Java is multi-paradigm because it natively supports object-oriented constructs (classes, interfaces, inheritance), imperative/procedural constructs (loops, static methods, mutable state) and functional constructs (lambdas, streams, Function/Supplier/Consumer interfaces) in one language.
- 3) Procedural → OO conversion:
Procedural:

```
static void printArea(int l, int w){ System.out.println(l*w); }
```


OO:

```
class Rectangle { int l, w; Rectangle(int l, int w){ this.l=l; this.w=w; } int area(){ return l*w; } void print(){ System.out.println(area()); } }
```

```
new Rectangle(4,5).print();
```

 — data and behavior are encapsulated in the object.